

Spring Boot — Lecture 5

Spring Core: IoC Container, Beans, @Component, @Autowired, @Bean

1. Recap — What We Did Without Spring

In Lecture 4 we saw Dependency Injection as a plain Java principle — we manually wired dependencies through Main.java. Now we let Spring Framework handle all of that automatically via its IoC Container.

```
// Lecture 4 approach – manual wiring in Main.java
PaymentService payment = new PaymentService();
OrderService order = new OrderService(payment); // manual DI
order.placeOrder();

// Lecture 5 goal – Spring does all this automatically
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
OrderService order = context.getBean(OrderService.class);
order.placeOrder(); // Spring already wired PaymentService inside
```

Spring's IoC Container replaces Main.java as the object creator and wirer. We just declare what we need — Spring handles the rest.

2. Adding Spring Core Dependency

We don't need Spring Boot or Spring MVC for this — we're building a console application. We only need **spring-context**, which brings in the IoC Container.

Dependency	When to use
spring-context	Spring Core IoC Container — all you need for DI, Beans, @Component etc. in a console app.
spring-boot-starter-web	Only needed for web/REST API applications. Includes Tomcat, Spring MVC etc.
spring-boot-starter	Spring Boot auto-configuration — overkill for a simple core demo.

pom.xml dependency to add:

```
org.springframework
spring-context
6.x.x
```

spring-context automatically pulls in spring-core, spring-beans, spring-aop and others as transitive dependencies. You don't need to declare them separately.

3. Java Reflection API — How Spring Works Internally

Before diving into Spring annotations, we need to understand the Reflection API because Spring uses it extensively to read class metadata at runtime.

The special Class<T> type:

```
// Normal object creation
Student s1 = new Student(); // s1 is an instance of Student

// Reflection – Class object holds metadata about Student
Class c1 = Student.class;

// c1 is NOT a Student instance – it holds Student's blueprint/metadata:
// - class name:    "Student"
// - fields:        String name, int age
// - constructors: Student()
// - methods:       getAttendance(), print()
```

```
// - annotations:    any @annotations on the class
// - access:         which members are public/private
```

What Reflection stores	Example from Student class
Class name	"Student"
Fields + types	String name, int age
Constructors	Student() — default constructor
Methods	getAttendance(), print()
Annotations	@Component, @Autowired etc. (Spring reads these)
Access modifiers	public, private, protected members

When Spring needs to create and manage an object, it uses Reflection to read the class blueprint — its constructor, fields, annotations — and then creates the object programmatically. This is how @Component and @Autowired work under the hood.

4. Two Ways to Configure Spring IoC Container

Approach	How it works	When used
Annotation-based (modern)	Use @Component, @Autowired, @Bean on classes. Spring scans and configures automatically.	All modern projects — this is what we learn first.
XML-based (legacy)	Define beans and wiring inside applicationContext.xml file with tags.	Old/legacy projects pre-2013. Still works but verbose and outdated.

Spring Boot makes annotation-based config even simpler — it auto-configures everything. But understanding the annotation approach first helps you understand what Spring Boot is doing behind the scenes.

5. @Component — Tell Spring to Manage This Class

@Component is a marker annotation. When placed on a class, it tells Spring: 'create and manage a Bean of this class in the IoC Container.'

```
// PaymentService.java
@Component
public class PaymentService {
    public void pay() {
        System.out.println("Payment done");
    }
}

// OrderService.java
@Component
public class OrderService {
    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

Spring will automatically create beans for both classes. It also detects that OrderService needs a PaymentService via its constructor and injects it automatically — no @Autowired needed with a single constructor.

6. AppConfig — Configuration Class

Spring needs a configuration class to know the rules: which package to scan for `@Component` classes. This class replaces what Main was doing manually.

```
// AppConfig.java
@Configuration          // marks this as a configuration class
@ComponentScan("in.coderarmyarmy") // scan this package for @Component
public class AppConfig {
    // empty for now – can add @Bean methods later
}
```

Annotation	Where it goes	What it does
@Configuration	On the AppConfig class	Tells Spring this class contains configuration rules. Spring reads it on startup.
@ComponentScan	On the AppConfig class	Tells Spring which package to scan for @Component classes. Scans sub-packages too.

@ComponentScan without a package argument scans the package where AppConfig lives. With an argument (e.g. 'in.coderarmyarmy'), it scans that package and ALL sub-packages recursively.

7. ApplicationContext — Starting the IoC Container

ApplicationContext is Spring's IoC Container interface. We start it in Main.java and pass our AppConfig class to it.

```
// Main.java
public class Main {
    public static void main(String[] args) {

        // Start the IoC Container with annotation-based config
        ApplicationContext context =
            new AnnotationConfigApplicationContext(AppConfig.class);
        //                                     ^
        //             AppConfig.class = metadata Spring needs to read rules

        // Fetch the managed bean from the container
        OrderService order = context.getBean(OrderService.class);

        // Call business method – Spring already injected PaymentService inside
        order.placeOrder();
        // Output:
        //   Payment done
        //   Order placed
    }
}
```

AnnotationConfigApplicationContext = use annotation-based configuration. We pass AppConfig.class (not AppConfig()) — Spring uses Reflection to read the metadata from the class itself.

8. IoC Container Startup — Step-by-Step Flow

Step	What Spring does
Step 1 — Start container	<code>new AnnotationConfigApplicationContext(AppConfig.class)</code> — IoC container starts up.
Step 2 — Read AppConfig	Spring reads AppConfig via Reflection. Sees <code>@Configuration</code> and <code>@ComponentScan</code> .
Step 3 — Component scan	Spring scans the given package (and sub-packages) for all classes with <code>@Component</code> .

Step	What Spring does
Step 4 — Find @Component classes	Finds: OrderService (@Component), PaymentService (@Component), etc.
Step 5 — Create Bean Definitions	BEFORE creating objects, Spring creates BeanDefinition metadata for each class: class name, scope, dependencies, annotations. Stored internally.
Step 6 — Create Bean objects	Spring creates actual objects (beans). PaymentService first (no dependencies). Then OrderService — detects it needs PaymentService, injects it via constructor.
Step 7 — Application uses beans	context.getBean(OrderService.class) returns the already-wired OrderService bean.

Bean Definitions are created before actual objects. This gives Spring a complete map of all classes and their dependencies before it starts instantiating anything — enabling correct dependency resolution order.

9. @Autowired and Types of Dependency Injection in Spring

@Autowired tells Spring to inject a dependency automatically. It can be placed on a constructor, setter, or field.

DI Type	@Autowired location	Code example	Recommended?
Constructor Injection	On the constructor	Not needed if only 1 constructor	YES — preferred approach
Setter Injection	On the setter method	@Autowired public void setPaymentService(PaymentService p)	OK for optional dependencies
Field Injection	Directly on the field	@Autowired private PaymentService paymentService;	NOT recommended — warning shown

Constructor injection (recommended — @Autowired optional):

```
@Component
public class OrderService {
    private final PaymentService paymentService; // final = good practice

    // @Autowired not required when only 1 constructor
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Setter injection:

```
@Component
public class OrderService {
    private PaymentService paymentService; // cannot be final

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Field injection (not recommended — IDE shows warning):

```
@Component
public class OrderService {
    @Autowired
    private PaymentService paymentService; // cannot be final, harder to test
}
```

```
}
```

Constructor injection advantages: (1) dependency wired at object creation time, (2) field can be final — no accidental reassignment, (3) easy to unit test — just pass a fake/mock via constructor without Spring.

10. Interfaces + Multiple Implementations

In real apps, services should be coded to interfaces. When multiple `@Component` classes implement the same interface, Spring gets confused about which one to inject.

```
// PaymentService.java – interface
public interface PaymentService {
    void pay();
}

// CardPayment.java
@Component
public class CardPayment implements PaymentService {
    public void pay() { System.out.println("Paying via Card"); }
}

// UpiPayment.java
@Component
public class UpiPayment implements PaymentService {
    public void pay() { System.out.println("Paying via UPI"); }
}
```

If both CardPayment and UpiPayment have @Component, Spring throws: NoUniqueBeanDefinitionException — expected single matching bean but found 2.

Fix 1 — @Primary (declare one as default):

```
@Component
@Primary // Spring picks this one when multiple beans match
public class UpiPayment implements PaymentService { ... }
```

Fix 2 — @Qualifier (specify exactly which bean to inject):

```
// Give each bean a qualifier name
@Component
@Qualifier("cp")
public class CardPayment implements PaymentService { ... }

@Component
@Qualifier("upi")
public class UpiPayment implements PaymentService { ... }

// In OrderService constructor – specify which one you want
@Component
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(@Qualifier("cp") PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Annotation	How it works	When to use
@Primary	Marks one bean as the default when multiple candidates exist	When you have a clear default and rarely need to switch
@Qualifier("name")	Specifies the exact bean by name at the injection point	When you need fine-grained control over which implementation is injected

Default bean name = class name in camelCase. CardPayment → 'cardPayment'. You can override with @Qualifier("customName") on the bean class.

11. @Bean — Creating Beans Manually

Sometimes you CANNOT use @Component on a class. Two common cases:

- The class has a complex constructor — Spring doesn't know what values to pass (e.g. a User class needing name and age).
- The class comes from a third-party library (compiled .class file) — you can't add @Component to it.

Solution: create the object yourself in AppConfig and give it to Spring via @Bean. Spring takes the returned object and manages it in the IoC Container.

```
// AppConfig.java
@Configuration
@ComponentScan("in.coderarmyarmy")
public class AppConfig {

    // Spring calls this method, gets the User object, stores it as a Bean
    @Bean
    public User createUser() {
        return new User("Aashutosh", 25); // we provide the values
    }

    // Same for third-party class – Spring manages the returned object
    @Bean
    public CartService createCartService() {
        return new CartService(); // CartService from external JAR
    }
}
```

Getting these beans in Main:

```
User user = context.getBean(User.class);
System.out.println(user.getName()); // Aashutosh

CartService cart = context.getBean(CartService.class);
cart.addToCart(); // Added to cart
```

Feature	@Component	@Bean
Where you put it	On the class itself	On a method inside @Configuration class
Who creates the object	Spring creates it automatically	You create it in the method body and return it
When to use	Your own classes with simple/no constructor	Complex constructors OR third-party library classes
Priority if both used	Lower priority	Higher priority — @Bean wins over @Component

12. Using @Bean With Interface Implementations

You can also replace @Component with @Bean for your own classes. This gives you full manual control over object creation and wiring.

```
// AppConfig.java – manually creating and wiring all beans
@Configuration
public class AppConfig {

    @Bean
    @Primary // or use @Qualifier
    public PaymentService createCardPayment() {
        return new CardPayment();
    }
}
```

```

    }

    @Bean
    public PaymentService createUpiPayment() {
        return new UpiPayment();
    }

    // Spring sees OrderService needs a PaymentService constructor arg.
    // It looks in the container – finds CardPayment (@Primary) and injects it.
    @Bean
    public OrderService createOrderService(PaymentService paymentService) {
        return new OrderService(paymentService);
    }
}

```

In @Bean methods, Spring auto-resolves method parameters from the IoC Container. If paymentService parameter has @Primary or @Qualifier set, Spring uses that. No @Autowired needed inside @Bean methods.

13. Why AppConfig? Why Not Just Use Main?

You could technically put all @Bean methods and configurations inside Main. But this is bad practice.

Main.java (Entry Point)	AppConfig.java (Configuration)
Should be clean and minimal — just starts the app	Holds all Spring configuration rules and @Bean definitions
Business developers read Main to understand app entry	Config changes happen in AppConfig without touching Main
Mixing config with entry point = messy, hard to maintain	Separation of concerns — config is separate from execution

AppConfig can be named anything — what matters is the @Configuration annotation on it. In Spring Boot, you won't even need AppConfig — Boot auto-configures everything.

14. BeanFactory vs ApplicationContext

Interface	Description	Status
BeanFactory	Original IoC container interface in early Spring. Basic bean creation and retrieval. Many methods now deprecated.	Legacy — rarely used directly
ApplicationContext	Super-interface that extends BeanFactory. Adds event publishing, i18n, AOP, web context support, and more. What we use.	Current standard — always use this
AnnotationConfigApplicationContext	Concrete implementation of ApplicationContext for annotation-based configuration.	What we instantiate in Main

ApplicationContext extends BeanFactory — everything BeanFactory does, ApplicationContext does too, plus much more. Always use ApplicationContext in your code.

Quick Reference — All Annotations Covered

Annotation	Where	Purpose
@Component	On a class	Mark class as a Spring-managed bean. Spring creates and manages its object.
@Configuration	On AppConfig class	Mark class as a configuration class. Spring reads its rules on startup.
@ComponentScan	On AppConfig class	Tell Spring which package to scan for @Component classes.
@Autowired	On constructor, setter, or field	Tell Spring to inject a dependency automatically. Optional for single constructor.
@Primary	On a @Component or @Bean class/method	Mark as default bean when multiple implementations of same type exist.

Annotation	Where	Purpose
@Qualifier("name")	On @Component or at injection point	Specify exact bean by name when multiple candidates exist.
@Bean	On a method inside @Configuration class	Manually create and return an object. Spring stores it as a managed bean.

Next Lecture: XML-based Configuration (legacy) — Bean scopes (Singleton vs Prototype)